

IMAGE EDITING SYSTEM

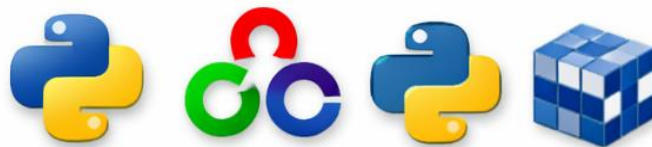
Khushi Kumari

Module: Computer Vision

NSTI Patna

Session: 2025 – 2026

Using Python, OpenCV & NumPy



Project Documentation

Name:- Khushi Kumari

Module:- Computer Vision

Q2. A photography company wants to create a simple image editing system.

The system should help to:

1. Change image brightness using thresholding
2. Remove noise using filters
3. Detect edges in images
4. Find important points/features in images
5. Join multiple images together to create a panorama

You need to implement these functions using Python, OpenCV, and NumPy.

Think about editing photos on a mobile app.

This project teaches the computer to:

- Convert images into black and white
- Blur images to remove unwanted noise
- Find object boundaries (edges)
- Detect special points in images
- Combine multiple photos into one wide photo

OpenCV helps us process images, and NumPy helps us work with image data.

Project Aim:-

To develop a **basic image editing system** for a photography company that can enhance and process photos by adjusting brightness, removing noise, detecting edges and features, and combining multiple images into a panorama.

Requirements:-

- **Programming Language:** Python
- **Libraries:**
 - OpenCV (cv2) → for image processing operations

- NumPy → for handling image arrays and numerical data

Features to Implement:-

1. Change Image Brightness using Thresholding

- Convert image to grayscale
- Apply cv2.threshold() or cv2.adaptiveThreshold()

2. Remove Noise using Filters

- Use cv2.GaussianBlur(), cv2.medianBlur(), or cv2.bilateralFilter()

3. Detect Edges in Images

- Apply cv2.Canny() for edge detection

4. Find Important Points/Features

- Use feature detectors like cv2.goodFeaturesToTrack() or cv2.ORB_create()

5. Join Multiple Images (Panorama)

- Use cv2.Stitcher_create() to combine images

Codes And Screenshot Output:-

```
import cv2

import numpy as np

# Load image in grayscale

img = cv2.imread("C:\\qqqq\\picturesque-mountain-valley-on-sunny-260nw-2477194071.webp", cv2.IMREAD_GRAYSCALE)

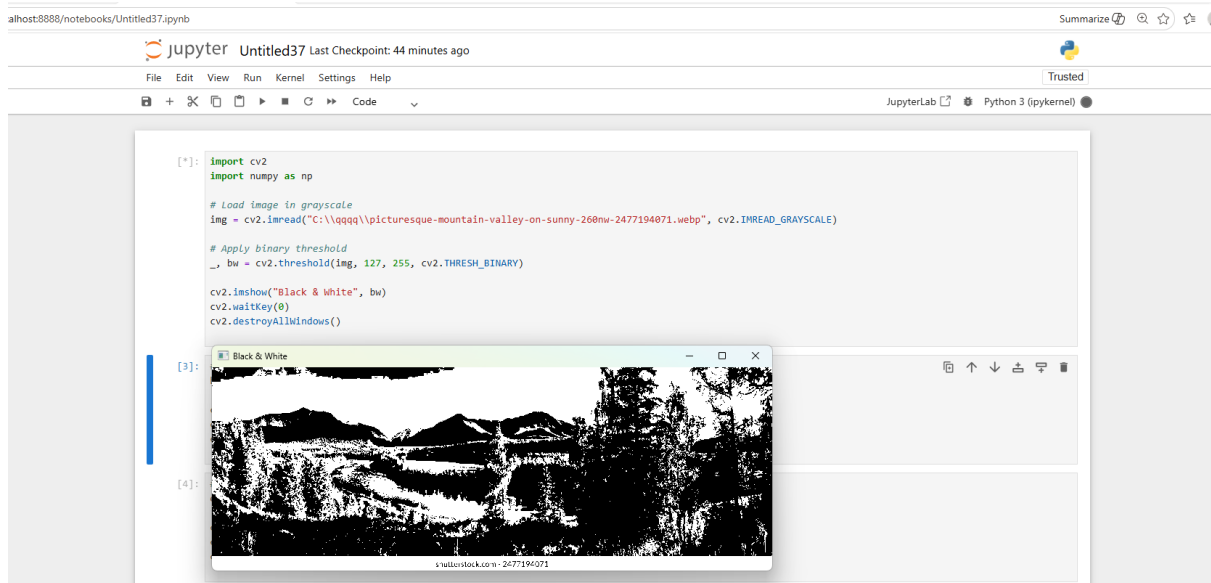
# Apply binary threshold

_, bw = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

cv2.imshow("Black & White", bw)

cv2.waitKey(0)

cv2.destroyAllWindows()
```



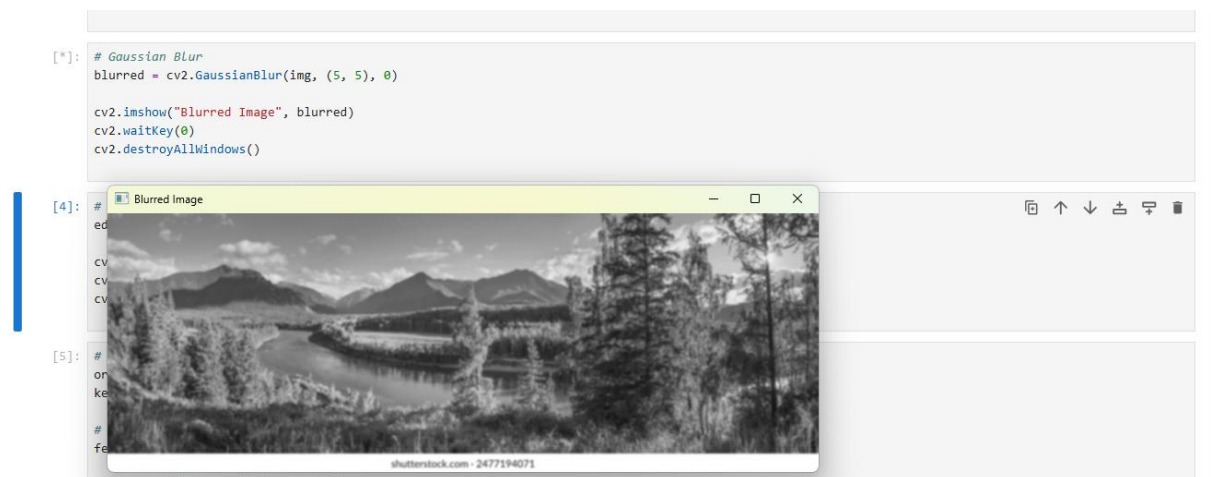
Gaussian Blur

```
blurred = cv2.GaussianBlur(img, (5, 5), 0)
```

```
cv2.imshow("Blurred Image", blurred)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```



Canny Edge Detection

```
edges = cv2.Canny(img, 100, 200)
```

```
cv2.imshow("Edges", edges)
```

```
cv2.waitKey(0)
```

`cv2.destroyAllWindows()`



Using ORB (Oriented FAST and Rotated BRIEF)

`orb = cv2.ORB_create()`

`keypoints, descriptors = orb.detectAndCompute(img, None)`

Draw keypoints

`feature_img = cv2.drawKeypoints(img, keypoints, None, color=(255,0,0))`

`cv2.imshow("Features", feature_img)`

`cv2.waitKey(0)`

`cv2.destroyAllWindows()`



```

# Load two overlapping images

img1 = cv2.imread(r"C:\\qqqq\\picturesque-mountain-valley-on-sunny-260nw-2477194071.webp\\left.jpg")

img2 = cv2.imread("C:\\qqqq\\picturesque-mountain-valley-on-sunny-260nw-2477194071.webp")

if img1 is None or img2 is None:

    print("Error: One or both images could not be loaded. Check file paths.")

else:

    # Optional: resize to same dimensions for consistency

    img1 = cv2.resize(img1, (800, 600))

    img2 = cv2.resize(img2, (800, 600))


# Stitcher API

stitcher = cv2.Stitcher_create()

(status, panorama) = stitcher.stitch([img1, img2])


if status == cv2.STITCHER_OK:

    cv2.imshow("Panorama", panorama)

    cv2.imwrite("panorama_output.jpg", panorama) # save result

    cv2.waitKey(0)

    cv2.destroyAllWindows()

else:

    print("Error during stitching. Status code:", status)

```

